

Reviewer Pack — Minimal Rank of Symmetric Functions over Tseitin Resolution ...

Entry bc2ea1f29a82 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 06:25:21 UTC

Minimal Rank of Symmetric Functions over Tseitin Resolution Trees Width

Entry ID: bc2ea1f29a82 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 06:25:21 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): Complexity Theory: Tseitin Resolution Trees

Statement:

[‘For a given Tseitin resolution tree T rooted at a variable x , the minimal rank of its associated symmetric function $f(T)$ is proportional to the width of T , specifically, $E[\text{rank}(f(T))] = c * \text{width}(T)$, where c is a constant.’, “Equivalently, for all Tseitin trees with width w on n variables, the Spearman’s rank correlation coefficient between $-\log_2(\text{rank}(f(T)))$ and $\log_2(w)$ across 30 random seeds is greater than or equal to some threshold α .”]

Rationale (proposer’s reasoning):

[‘Symmetric functions capture algebraic properties of polynomials that are invariant under permutations, a property relevant in the analysis of polynomial operations used in resolution proof systems. The width of Tseitin trees is a measure of their depth and complexity. A connection between these two would provide insights into the structure of resolution proofs.’, ‘Previous work has shown that symmetric functions can be used to analyze algebraic properties of polynomials, while the width of Tseitin trees is directly related to the size of the proof required for SAT problems. This conjecture posits a quantitative relationship between these concepts.’]

Taxonomy category: SymmetricFunctionTheory × TseitinResolutionTrees (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6ddc1c832c67cdae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman’s rank correlation coefficient (CRC) between $-\log_2(\text{rank}(f(T)))$ and $\log_2(\text{width}(T))$ across 30 random seeds is greater than or equal to a predefined threshold α .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symmetric function theory AND Tseitin resolution trees width AND minimal rank - Tseitin resolution trees AND symmetric functions AND rank analysis - complexity theory AND Tseitin resolution AND symmetric function evaluation

Top relevant hits considered: - [<http://arxiv.org/abs/2311.03041v2>] On the unitary representation theory of locally compact contraction groups - [<http://arxiv.org/abs/math/0609339v3>] On distinguishing trees by their chromatic symmetric functions - [<http://arxiv.org/abs/1902.01595v1>] Integral means inequalities, convolution, and univalent functions - [<http://arxiv.org/abs/astro-ph/0208243v7>] Measurement of the Flux of Ultrahigh Energy Cosmic Rays from Monocular Observations by the High Resolution Fly's Eye Exp - [<http://arxiv.org/abs/astro-ph/0407622v3>] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/0911.3482v5>] Complexity of Networks (reprise) - [<http://arxiv.org/abs/nlin/0101006v1>] On Complexity and Emergence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_tseitin_tree(n):
    if n == 1:
        return (None, None, 'x')
    else:
        left_size = random.randint(1, n-2)
        right_size = n - left_size - 1
        left = generate_tseitin_tree(left_size)
        right = generate_tseitin_tree(right_size)
        return (left, right, f'¬({generate_tseitin_tree(left_size)[2]} ∨ {generate_tseitin_tree(right_size)[2]})')

def calculate_rank(poly):
    # Placeholder for calculating the rank of a polynomial
    # This is a dummy implementation and should be replaced with actual logic
    return len(poly)

def run_trial(seed: int) -> dict:
```

```

random.seed(seed)
n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    if n * (n - 1) // 2 < 1: # Avoid empty range in randrange
        continue

    width_sum = 0
    rank_sum = 0
    instances_tested = 0

    for _ in range(5): # Ensure at least 30 instances per seed
        tree = generate_tset_in_tree(n)
        poly = calculate_rank(tree) # Placeholder for actual polynomial calculation
        width = len(tree[2].split(' ')) - 1
        rank = calculate_rank(poly)

        width_sum += width
        rank_sum += rank
        instances_tested += 1

    avg_width = width_sum / instances_tested
    avg_rank = rank_sum / instances_tested
    crc = spearman_correlation([-math.log2(avg_rank), math.log2(avg_width)])

    results.append({
        "metric_name": "Spearman's Rank Correlation",
        "metric_value": crc,
        "instances_tested": instances_tested,
        "conjecture_holds": crc >= 0.8, # Placeholder threshold
        "counterexample": ""
    })

return {
    "seed": seed,
    "results": results
}

def spearman_correlation(x):
    n = len(x)
    if n < 2:
        raise ValueError("At least two data points are required")

    sorted_x = sorted((x[i], i) for i in range(n))
    rank_x = [sorted_x.index(i[1]) for i in sorted_x]

    sorted_y = sorted((y, i) for i, y in enumerate(x))
    rank_y = [sorted_y.index(i[1]) for i in sorted_y]

    d_squared_sum = sum((rank_x[i] - rank_y[i]) ** 2 for i in range(n))
    spearman_coefficient = 1 - (6 * d_squared_sum) / (n * (n**2 - 1))

    return spearman_coefficient

if __name__ == "__main__":

```

```

import sys
seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result["results"])

all_crcs = [r[0]["metric_value"] for r in results]
mean_crc = sum(all_crcs) / len(all_crcs)
std_crc = math.sqrt(sum((x - mean_crc) ** 2 for x in all_crcs) / len(all_crcs))
support_fraction = sum(1 for r in results if r[0]["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_crc} std={std_crc} support_fraction={support_fraction}")
elif any(not r[0]["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result[0]["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e39459ce.py", line 95, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e39459ce.py", line 47, in run_trial

```

    tree = generate_tseitlin_tree(n)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e39459ce.py", line 24, in generate_tseitlin_tree

```

    left = generate_tseitlin_tree(left_size)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e39459ce.py", line 22, in generate_tseitlin_tree

```

    left_size = random.randint(1, n-2)
    ~~~~~

```

File "/usr/lib/python3.12/random.py", line 336, in randint

```

    return self.randrange(a, b+1)
    ~~~~~

```

File "/usr/lib/python3.12/random.py", line 319, in randrange

```

    raise ValueError(f"empty range in randrange({start}, {stop})")

```

ValueError: empty range in randrange(1, 1)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Spearman’s rank correlation coefficient and thus confirming or refuting the conjecture. | next: Investigate the cause of the crash in the test code and run the test again to obtain the necessary data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12978
2	preregistration	ollama_remote	glm4:latest	0	0	5633
3	novelty	ollama_remote	glm4:latest	0	0	4610
4	novelty	ollama_remote	glm4:latest	0	0	6462
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16864
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7802
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15325
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11742
9	judge	ollama_remote	glm4:latest	0	0	15746

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97161 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/bc2ea1f29a82.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bc2ea1f29a82.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bc2ea1f29a82.tar.gz (if generated)